1. LIVE MIDTERM EXAM (FALL 2026 LIVE PAPER) — COMPLETE SOLUTIONS & PEDAGOGICAL TRACING TABLES

Official Paper Breakdown · Q1 (50%) + Q2 (50%) · Zero Blank Space

Q1: 6-PART 1D ARRAY COMPLETE PROGRAM WITH PEDAGOGICAL SOLUTION COMMENTS — 50% WEIGHT

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main() {
  // a) Task: Input 1D array A of 10 doubles from keyboard:
  double A[10];
  for (int i = 0; i < 10; i++) cin >> A[i];

  // b) Task: Minimum element & index -> MUST init minVal = A[0] (fails if 0.0):
  double minVal = A[0]; int minIdx = 0;
  for (int i = 1; i < 10; i++) {
    if (A[i] < minVal) { minVal = A[i]; minIdx = i; }
  }
  cout << "Min: A[" << minIdx << "] = " << minVal << "\n";

  // c) Task: Check all elements in range [-7, 7]; else terminate immediately:
  bool inRange = true;
  for (int i = 0; i < 10; i++) {
    if (A[i] < -7.0 || A[i] > 7.0) { inRange = false; break; }
  }
  if (inRange) cout << "elements of A are in range\n";
  else return 0; // Immediate halt

  // d) Task: Repeatedly input i1, i2 until valid (0..9 and i1 != i2):
  int i1, i2;
  do {
    cin >> i1 >> i2;
  } while (i1 < 0 || i1 > 9 || i2 < 0 || i2 > 9 || i1 == i2);

  // e) Task: Swap elements i1 and i2 of A -> MUST use temp to avoid overwrite:
  double temp = A[i1]; A[i1] = A[i2]; A[i2] = temp;

  // f) Task: Save elements of A to "A.txt" -> open ofstream, verify, write, close:
  ofstream fout("A.txt");
  if (!fout) return 1;
  for (int i = 0; i < 10; i++) fout << A[i] << "\n";
  fout.close();
  return 0;
}
```

Q2: UNIVERSAL TRACING PROTOCOL & CRITICAL EXAM CORE CONCEPTS — 50% WEIGHT

- **1. Truncating Integer Div:** `1/(0+3)=1/3=0` (NOT 0.33!). `7/2=3`. Float div: `7.0/2=3.5`.
- **2. IEEE-754 Div by Zero:** `7.0/(1-1)=7.0/0.0=inf` → **Does NOT crash!** Float div yields `inf`.
- **3. Logical NOT `!i`:** `!0=1 (true)`; `!1=!2=0 (false)`. `else if (!i)` runs ONLY at *i=0*!
- **4. Uninitialized Garbage (G):** `int x;` holds undefined RAM garbage. `x += B[i];` stays **Garbage (G)**!
- **5. Loop Variable Retention & Bounds:** After `for(i=0;i<N;i++)`, *i* retains **N**! Reading `A[N]` is out-of-bounds → **Garbage (G)**!
- **6. Pre-Break Trap:** Statements before `if(cond) break;` execute even on the terminating pass!
- **7. Modulo % Rules:** `20 % 7 = 6`. Sign follows LHS: `-20 % 7 = -6`; `20 % -7 = +6`.
- **8. Semicolon Trap:** `for(i=0;i<5;i++);` loops empty 5 times; body runs ONCE with *i=5*!

Pass-by-Reference vs Array Decay:
- Swap by Ref: `void swap(double &a, double &b){ double t=a; a=b; b=t; }` (modifies caller!)
- Array Decay: `void f(double A[], int n)` (passes pointer, always pass size *n*)
- Pass by Value: `void f(int x)` (local copy; caller unaffected)

File I/O Streams (<fstream>):
- Read EOF Loop: `ifstream fin("in.txt"); if(!fin) return 1; while(fin >> A[n] && n<MAX) n++; fin.close();`
- Write File: `ofstream fout("A.txt"); for(int i=0;i<n;i++) fout<<A[i]<<"\n"; fout.close();`

C-Strings & Space Stripper:
- Null Terminator: `char s[20];` must end with `'\0'`. `strlen(s)` excludes `'\0'`.
- Space Removal: `int j=0; for(int i=0;s[i]!='\0';i++) if(s[i]!=' ') s[j++]=s[i]; s[j]='\0';`

Universal Tracing Table Protocol:
- Columns: `i | Condition | Variables that Change | Output`
- Rule: Only record changed values (Gordon shortcut).
- Box Answers: Always draw a clean box around final outputs inside answer booklet!

2. ALL SIMILAR EXAM PROBLEMS & ALGORITHMS ACROSS PAST GORDON MIDTERMS & QUIZZES

Fall 2023 · Fall 2022 · Quiz 1 & 2 · Reviews 1 & 4 · Exact Twin Match

1. VALIDATED SIZE N & 2 ARRAYS — F22 Q1A / REV 4

```cpp
// Task (a): Input two 1D arrays of doubles (size n) with validation:
int n; cin >> n;
while (n < 1 || n > 100) {
  cout << "Re-enter n (1-100): "; cin >> n;
}
double A[100], B[100];
for (int i = 0; i < n; i++) cin >> A[i];
for (int i = 0; i < n; i++) cin >> B[i];
```

2. VECTOR DOT PRODUCT — F22 Q1B / QUIZ 2

```cpp
// Task (b): Calculate dot product of A and B (sum += A[i]*B[i]):
double dot = 0.0;
for (int i = 0; i < n; i++) dot += A[i] * B[i];
cout << "Dot product = " << dot << "\n";
// Pos average: if (A[i] > 0) { s += A[i]; cnt++; }
avg = s / cnt;
```

3. STRICT RANGE FILTER (0, PI) — F22 Q1C

```cpp
// Task (c): Count elements of B strictly between 0 and PI:
const double PI = 4.0 * atan(1.0);
int count = 0;
for (int i = 0; i < n; i++) {
  if (B[i] > 0.0 && B[i] < PI) count++;
}
cout << "Elements in (0, PI) = " << count << "\n";
```

4. ARRAY MERGE / CONCATENATE — F22 Q1D

```cpp
// Task (d): Combine arrays A and B into single array C of size 2n:
double C[200];
for (int i = 0; i < n; i++) {
  C[i]     = A[i]; // 1st half: indices 0..n-1
  C[i + n] = B[i]; // 2nd half: indices n..2n-1
}
```

5. SENTINEL LOOP & POSITIVES — F22 Q1E

```cpp
// Task (e): Input k until negative (sentinel break); print pos count & sum:
int k, posCount = 0, posSum = 0;
while (true) {
  cin >> k;
  if (k < 0) break;
  if (k > 0) { posCount++; posSum += k; }
}
cout << "Pos count=" << posCount << ", Sum=" << posSum << "\n";
```

6. IN-PLACE ARRAY REVERSAL — F23 Q1C

```cpp
// Task: Swap elements symmetrically up to n/2 (zero extra memory):
for (int i = 0; i < n / 2; i++) {
  double temp = A[i];
  A[i] = A[n - 1 - i];
  A[n - 1 - i] = temp;
}
// Traverses only first half (n/2); avoids double-swapping!
```

7. CYCLIC FORWARD SHIFT — F22 Q1D / REV 4

```cpp
// Task: Shift forward: wrap last element A[n-1] to 0, shift others right:
double A_sh[100];
A_sh[0] = A[n - 1]; // Wrap last element
for (int i = 0; i < n - 1; i++) A_sh[i + 1] = A[i];
// In-place: double last = A[n-1]; for(i=n-1;i>0;i--) A[i]=A[i-1]; A[0]=last;
```

8. ARRAY EQUALITY TEST — MIDTERM REV 1

```cpp
// Task: Check if A and B equal: assume true, break on first mismatch:
bool equal = true;
for (int i = 0; i < n; i++) {
  // Doubles tolerance: if (fabs(A[i] - B[i]) > 1e-7)
  if (A[i] != B[i]) { equal = false; break; }
  // TRAP: NEVER add 'else equal=true;' inside loop!
}
if (equal) cout << "Equal"; else cout << "Not equal";
```

9. MENU LOOP ALL-OPTION FLAG — QUIZ 2 / REV 1

```cpp
// Task: Repeat menu until every option (1=Avg, 2=Min, 3=Dot) chosen >= 1 time:
bool used[4] = { false }; int ch;
do {
  cout << "1=Avg, 2=Min, 3=Dot: "; cin >> ch;
  if (ch == 1) { /* avg */ used[1] = true; }
  else if (ch == 2) { /* min */ used[2] = true; }
  else if (ch == 3) { /* dot */ used[3] = true; }
} while (!(used[1] && used[2] && used[3]));
```

3. ESSENTIAL LIBRARIES, NUMERICAL CONSTANTS & FUNCTION / FILE STREAM TEMPLATES

Directives & Memory Reference · No Fluff

DIRECTIVES & HEADERS — BOILERPLATE

- `#include <iostream>` (cin, cout)
- `#include <cmath>` (sqrt, pow, fabs, sin)
- `#include <fstream>` (ifstream, ofstream)
- `#include <iomanip>` (fixed, setprecision)
- `#include <cstdlib>` (exit(0), abs)
- `using namespace std;`

MATH & FORMATTING — FORMULAS

Concordia PI Constant:
`const double PI = 4.0 * atan(1.0);`
Float Equality (NEVER ==):
`if (fabs(a - b) < 1.0e-7) { ... }`
Decimal Output (2 places):
`cout << fixed << setprecision(2);`

OPERATOR PRECEDENCE — ORDER

1. `() [] x++ x--`
2. `++x --x ! (type) sizeof` (R-to-L)
3. `* / %` 4. `+ -`
5. `<< >>` 6. `< <= > >=`
7. `== !=` 8. `&&` 9. `||`
10. `= += -= *= /= %=` (R-to-L)

DYNAMIC MEMORY & POINTERS — ZERO LEAK

Dynamic Array Alloc & Delete:
`double *A = new double[n];`
`if (!A) return 1; /* check allocation */`
`delete[] A; A = nullptr; /* free RAM */`
Pointer Dereference: `*p` accesses value; `&x` gets address.